

run_oc_simple_time_sim_arbitrary(25).m
檔案
my_model(26).m
檔案
my_params(35).m
檔案

Given simulation data from the current model with/without added noise or from Carsim, for a given lateral maneuver such as in the code and the tie duration of concern, for example, after steering happens, how to determine a constant d or time varying d(t) so that Fz, or Fx & Fy, or Fz, Fx and Fy of each tyre can be fitted by the current model as good as possible? The fitting objective can be a time-weighted tuning option so that either immediate next steps or longer future fitting effect can be emphasized. I ask these questions because in the future, I would like to try to implement the current vehicle model in NMPC for extreme maneuvers such as transient drifting and lap simulation, and I think to predict the tyre load and tyre forces in the prediction horizon

顯示更多

I'll treat this as a parameter/state-estimation problem, not just a curve fit. The key is to separate "can the data identify d?" from "what optimizer should estimate it," because tyre-load fitting and tyre-force fitting have different sensitivity to d.

I've confirmed the important structure: in your code, d is not an isolated post-processing gain. It sets q_1 = q_2 = d and also changes H = h - d , so it changes both lateral load-transfer equations and the roll/yaw/pitch dynamics.

已思考 2m 46s

Yes, this is possible, but I would not start by fitting arbitrary time-varying $d(t)$ directly. I would do it in this order:

- constant scalar d
- constant front/rear effective heights q_1 , q_2
- slowly varying or state-scheduled $d(x,u)$ / $q_1(x,u)$, $q_2(x,u)$
- only then consider free $d(t)$ as a diagnostic, not as an NMPC-ready model

In your current parameters, `veh.d` is set as `0.7*h`, then `veh.H` = `h-d`, and `veh.q1` = `veh.q2` = `d`, so one scalar d simultaneously changes the lateral load-transfer moment arms and the roll/yaw/pitch dynamics through `H` `my_params`. In `my_model.m`, `H` enters `theta_ddot`, `phi_ddot`, `r_dot`, `ux_dot`, and `uy_dot`, while `q1` and `q2` enter the front and rear lateral load-transfer equations directly `my_model`. Your simulation code already exposes the needed model outputs: `f_h_load`, `f_load`, and `f_tyre`, and post-processing stores `slm_z_phys` and `slm_tyre` `run_oc_simple_time_sim_arbitrary`.

1. First distinguish three fitting problems

Problem A: fit F_z only

This is the cleanest and most physically meaningful way to identify d .

Your lateral load equations are essentially:

$$\begin{aligned}\Delta Z_1 &= \frac{Y_1 q_1 + k_{\phi 1} \dot{\phi}}{t_1} \\ \Delta Z_2 &= \frac{Y_2 q_2 + k_{\phi 2} \dot{\phi}}{t_2}\end{aligned}$$

with

$$\begin{aligned}F_{z,fr} &= 0.5Z_1 + \Delta Z_1, & F_{z,fl} &= 0.5Z_1 - \Delta Z_1 \\ F_{z,rr} &= 0.5Z_2 + \Delta Z_2, & F_{z,rl} &= 0.5Z_2 - \Delta Z_2\end{aligned}$$

If you assume q_1 = q_2 = d , then the most direct diagnostic estimate is:

$$\begin{aligned}a_{1,i} &= t_1 \frac{F_{z,fr,i}^{data} - F_{z,fl,i}^{data}}{2} - k_{\phi 1} \dot{\phi}_i - c_{\phi 1} \dot{\phi}_i \\ a_{2,i} &= t_2 \frac{F_{z,rr,i}^{data} - F_{z,rl,i}^{data}}{2} - k_{\phi 2} \dot{\phi}_i - c_{\phi 2} \dot{\phi}_i\end{aligned}$$

and ideally:

$$a_{1,i} \approx Y_{1,i} d, \quad a_{2,i} \approx Y_{2,i} d$$

So a weighted least-squares constant estimate is:

$$\hat{d} = \frac{\sum_i w_i \left(Y_{1,i} a_{1,i} / \sigma_1^2 + Y_{2,i} a_{2,i} / \sigma_2^2 \right)}{\sum_i w_i \left(Y_{1,i}^2 / \sigma_1^2 + Y_{2,i}^2 / \sigma_2^2 \right)}$$

This is a very useful first test because it tells you whether a **single constant d is even plausible**.

If the instantaneous estimates from front and rear agree and stay nearly constant after steering, then scalar d is identifiable. If the front and rear estimates disagree, you should estimate q_1 and q_2 separately. If both vary strongly with time, then scalar d is compensating for missing suspension/tyre/load-transfer dynamics.

Problem B: fit F_x and F_y only

This is much weaker for identifying d .

d affects F_x and F_y mainly indirectly through F_z , and then through the tyre model. If the tyres are in a near-linear slip region, F_x / F_y may not be very sensitive to F_z . In that case, many different d values can produce almost the same F_x / F_y .

So I would **not** identify d from F_x, F_y alone unless:

- the manoeuvre reaches high lateral acceleration or combined slip,
- the tyre model's load sensitivity is accurate,
- tyre parameters are already trusted,
- and F_z is unavailable.

Otherwise, fitting d to F_x, F_y may just use d to hide tyre-model errors.

Problem C: fit F_z , F_x , and F_y together

This is useful for **validation** or **joint tuning**, but not as the first identification method.

Use a normalized weighted objective:

$$J(d) = \sum_i w_i \left[\lambda_z \frac{\hat{F}_{z,i}(d) - F_{z,i}^{data}}{\sigma_z}^2 + \lambda_x \frac{\hat{F}_{x,i}(d) - F_{x,i}^{data}}{\sigma_x}^2 + \lambda_y \frac{\hat{F}_{y,i}(d) - F_{y,i}^{data}}{\sigma_y}^2 \right]$$

Do not use raw Newtons directly without normalization. Otherwise the largest-magnitude channel dominates the fit.

My recommended order is:

identify d mainly from $F_z \rightarrow$ validate using F_z, F_y

not the opposite.

2. Use two different objectives: one-step output fit and horizon simulation fit

For your NMPC purpose, you need both.

One-step output-error objective

At each sampled time, use the data state and control:

$$x_i^{data}, \quad u_i^{data}, \quad \kappa_i$$

Then evaluate your model with candidate d :

$$\hat{y}_i(d) = g(x_i^{data}, u_i^{data}, \kappa_i, d)$$

where y can be F_z , F_x, F_y , or all tyre forces.

This isolates the load-transfer parameter from state-trajectory mismatch. For clean current-model data, this should recover the original d almost exactly. If it does not, likely causes are wrong sign convention, using commanded instead of actual controls, not resolving the algebraic F_z loop for each candidate d , or mismatched force frames.

Multi-step simulation-error objective

For NMPC relevance, also do:

$$\hat{x}_{i+1}, \dots, \hat{x}_{i+N} = \text{simulate current model from } x_i^{data} \text{ using candidate } d$$

Then compare outputs over a 2–3 s horizon:

$$J_{horizon}(d) = \sum_{j=0}^N w_j \left(W_y \left(\hat{y}_{i+j}(d) - y_{i+j}^{data} \right)^2 \right)$$

This tells you whether the chosen d is useful for prediction, not just instantaneous force matching.

For NMPC, this second objective is more important, but I would still calibrate first with the one-step objective because it is easier to debug.

3. Time weighting for “immediate next step” versus “longer future”

For a window starting at t_0 , define $j = 0, \dots, N$.

To emphasize immediate prediction:

$$w_j = \exp\left(-\frac{j\Delta t}{\tau}\right)$$

with small τ , for example 0.2–0.5 s.

To emphasize the whole 2–3 s MPC horizon:

$$w_j = 1$$

or use a very slow decay.

To emphasize later horizon behavior:

$$w_j = \left(\frac{j}{N}\right)^p$$

with $p = 1$ or 2, though I would use this carefully because the early part of the prediction is usually more important for closed-loop control.

A practical objective is:

$$J(d) = \sum_{j=0}^N w_j \left(W_y \left(\hat{y}_j(d) - y_j^{data} \right)^2 + \lambda_d (d - d_{nom})^2 \right)$$

with

$$0 \leq d \leq h$$

or a slightly tighter bound such as:

$$0.05h \leq d \leq 0.95h$$

4. How to estimate constant d

For a scalar constant d, do not overcomplicate it.

Use this workflow:

1. Choose the fitting window, for example from steering start to steering start + 2-3 s.
2. Use actual applied controls, not just command histories.
3. Use measured/simulated states and tyre outputs.
4. Define normalized residuals.
5. Sweep d over a grid, for example $d/h = 0.1, 0.15, \dots, 0.95$.
6. Plot $J(d)$.
7. Then refine with `fminbnd` or `lsqnonlin`.

For a single scalar, the grid plot is extremely valuable. You want to see whether the objective has a clear minimum. If $J(d)$ is flat, then the manoeuvre does not identify d.

A clean current-model test should behave like this:

$$d_{true} \rightarrow \text{generate clean data} \rightarrow \text{fit } d \rightarrow \hat{d} \approx d_{true}$$

If this fails on noise-free data, then the identification implementation is wrong.

5. How to estimate time-varying d(t)

A free value of d at every time sample is dangerous. It will overfit noise and unmodelled effects.

Use a smooth parameterization:

$$d(t) = \sum_m B_m(t) c_m$$

where $B_m(t)$ are B-spline or piecewise-linear basis functions.

Then solve for the coefficients c_m :

$$J(c) = \sum_i w_i \left(\hat{y}_i(d(t_i)) - y_i^{data} \right)^2 + \lambda_1 \sum_m (c_m - d_{nom})^2 + \lambda_2 \sum_m (c_{m+1} - c_m)^2 + \lambda_3 \sum_m (c_{m+1} - 2c_m + c_{m-1})^2$$

with bounds:

$$0 \leq d(t) \leq h$$

This gives a smooth effective d(t). But interpret it carefully:

$d(t)$ is not necessarily a physical roll-center height.

It may be compensating for missing suspension kinematics, tyre relaxation, camber effects, roll-center migration, compliance, CarSim suspension details, or tyre-model mismatch.

For NMPC, a purely time-varying d(t) is not very useful unless the future time profile is known. A better NMPC-compatible result is a state-scheduled model:

$$d = d(x, u)$$

for example:

$$d = d_0 + d_{ay} |a_y| + d_\phi \phi + d_\dot{\phi} \dot{\phi}$$

or, better, estimate front/rear separately:

$$q_1 = q_{10} + q_{1,ay} |a_y| + q_{1,\phi} \phi$$

$$q_2 = q_{20} + q_{2,ay} |a_y| + q_{2,\phi} \phi$$

This can be evaluated inside the NMPC prediction horizon because it depends on predicted states, not unknowable future measurement data.

6. What simulations I would run

For identifying d, do not start from drifting.

Use a sequence:

Stage 1: clean current-model data

Use your own model with known veh.d.

Run:

- step steer, multiple steering amplitudes,
- sine steer, multiple frequencies,
- linear ramp steer,
- several speeds,
- zero torque after steering, as you currently do,
- later, combined braking/steering and throttle/steering.

Goal: prove the estimator can recover the known d.

Stage 2: noisy current-model data

Add realistic noise to:

- F_x, F_y ,
- F_x, F_y ,
- u_x, u_y, r ,
- $\phi_1, \phi_1 \dot{}, \theta, \theta \dot{}$,
- steering angle.

Then check estimator variance and bias.

Stage 3: CarSim data

Use CarSim as "truth," but expect the best-fit d to depend on:

- manoeuvre,
- speed,
- lateral acceleration,
- tyre slip level,
- suspension state,
- whether you fit F_z only or all tyre forces.

If the CarSim-fitted d varies strongly with manoeuvre, do not force one scalar d. Move to q1,q2 or a state-scheduled model.

Stage 4: extreme manoeuvres

Only after the above works, test:

- transient drift entry,
- high slip angle,
- combined throttle/steering,
- braking while steering,
- aggressive lap-simulation corner entry.

For these cases, tyre model mismatch may dominate d mismatch. So use d fitting as one part of a broader validation, not as the only tuning knob.

7. Specific recommendation for your NMPC goal

For NMPC over 2-3 s, I would use this structure:

offline identify robust constant or scheduled $d/q_1/q_2$

then:

online MHE estimates small correction Δd

then NMPC predicts with:

$$d_{MPC} = d_{schedule}(x, u) + \Delta d_{MHE}$$

Do not let NMPC use an arbitrary future d(t) unless you have a causal dynamic model for it. Otherwise it becomes non-predictive.

My strongest recommendation is:

Start with F_z -only identification of constant d.

Then check:

Does that same d also improve F_x, F_y prediction?

If yes, scalar d is useful.

If no, do not keep tuning d to fit F_x, F_y ; investigate tyre model, combined-slip behavior, camber, relaxation, and suspension/load-transfer structure.

For your current code, the best next implementation step is to promote d from a hard-coded value in `my_params.m` into a CasADi parameter, for example:

$$pv = [\kappa; d]$$

then inside `my_model.m` use:

$$H = h - d, \quad q_1 = d, \quad q_2 = d$$

or, better for testing:

$$pv = [\kappa; H; q_1; q_2]$$

This avoids rebuilding the CasADi model for every candidate d and makes `lsqnonlin`, grid search, MHE, or even future NMPC adaptation much easier.

Please double check your answer to make sure every point in your answer is rigorous and worthy to try.

run_oc_simple_time_sim_arbitrary(26).m
檔案

my_model(27).m
檔案

my_params(36).m
檔案

Then can you modify my codes here in order to carry out your suggested point 1, 2, 3, 4, 5 and 6? Make sure you leave the current simulations untouched and give me the options for identification (so I can choose whether to do simulations alone as the codes right now, or after a certain simulation I can also carry out identification).

I'll audit the modified package directly, not just reason from memory. I'll check three things: original simulation preservation, `pv=[kappa;H;q1;q2]` consistency, and whether the six identification steps are actually implemented without breaking default runs.

So far the default path looks structurally preserved: the original `ode15s`, fixed RK4, optional `ode15i`, post-processing, diagnostics, and plotting functions are still present. The unavoidable change is that every model evaluation now receives a 4-entry parameter vector instead of scalar `kappa`, with defaults chosen to reproduce your original `veh.H`, `veh.q1`, and `veh.q2`.

I found and fixed one subtle issue during the audit: for legacy profiles, the identification window could have defaulted to the phased-profile metadata time instead of the first real steering input. I patched it so phased profiles use `ctrl.t_steer_start`, while legacy profiles detect the first nonzero steering command.

已思考 2m 24s ›

I re-audited the package and patched one subtle issue I found. The updated package is here:

[Download updated full package](#)

Individual updated files:

[run_oc_simple_time_sim_arbitrary.m](#)
[my_model.m](#)
[my_params.m](#)
[README_IDENTIFICATION_USAGE.md](#)

Audit result

1. Original simulation functions are still there

Confirmed. The original simulation structure is still present:

- `ode15s` reduced-ODE simulation
- fixed-step RK4 simulation
- optional `ode15i` DAE simulation
- `postprocess_solution`
- `postprocess_dae_solution`
- command-vs-actual plotting
- pre-steering diagnostics
- main result plotting

The default remains:

```
MATLAB
```

So if you run the code as before, identification does **not** run.

The only unavoidable simulation-path change is that model evaluation now passes:

```
MATLAB
```

instead of only:

```
MATLAB
```

But the default values are:

```
H = veh.H;  
q1 = veh.q1;
```

```
MATLAB
```

so the nominal simulation physics should be unchanged.

2. `my_params.m` is unchanged

I checked it directly. The provided `my_params.m` is byte-for-byte unchanged relative to your uploaded file.

So your original parameter definitions remain:

```
veh.d = veh.h * 0.7;  
veh.H = veh.h - veh.d;  
veh.q1 = veh.d;
```

```
MATLAB
```

3. `my_model.m` was changed only where necessary

The only meaningful changes in `my_model.m` are:

```
H_pv = SX.sym('H_pv');  
q1_pv = SX.sym('q1_pv');  
q2_pv = SX.sym('q2_pv');
```

```
MATLAB
```

and then:

```
MATLAB
```

```
Out1=71 - /V+et m; a uah bwhct+abdt a uah bwhct+abdt+dx) / uah 85 +
```

```
MATLAB
```

That is exactly the intended change: make `H`, `q1`, and `q2` runtime-identifiable without rebuilding the CasADi graph.

4. Identification points 1–6 are implemented

The implemented workflow is:

1. Use clean current-model data from one completed simulation.
2. Fit free $[H,q1,q2]$ using nonlinear least squares through `fminsearch`.
3. Check whether single scalar `d` is supported, using:

$$q_1 \approx q_2, \quad H \approx h - \frac{q_1 + q_2}{2}$$

4. Fit scalar `d` using:

$$H = h - d, \quad q_1 = q_2 = d$$

through `fminbnd`.

5. Recommend whether to keep scalar `d` or free $[H,q1,q2]$.
6. Run optional noise robustness / Monte Carlo identification.

5. I fixed one subtle identification-window issue

In the first version I gave you, if you used a legacy profile such as `m11d_sine`, the identification window could default to `ctrl.t_steer_start`, even though legacy steering starts earlier.

I fixed that.

Double check again, because every time I ask you to check you seem to find some issues. So make sure you find all issues at once, do not let me ask you time after time

You are right to be concerned. I'm going to stop doing small spot-checks and instead run a systematic audit pass: file existence, syntax-risk scan, old/new diff, all `pv` call sites, all identification functions, and package consistency.

The broader audit is now finding design-level issues rather than just syntax issues. I've confirmed the default simulation branch still builds and calls the same solver functions, but I'm checking whether the identification objective would truly recover the original parameters from clean self-generated data.

已思考 3m 42s >

You are right. I did a broader audit this time instead of another spot-check. I found two more issues/design weaknesses and patched them in the package.

Use this newest package, not the previous one:

[Download final rechecked package](#)

Individual files:

[run_oc_simple_time_sim_arbitrary.m](#)
[my_model.m](#)
[my_params.m](#)
[README_IDENTIFICATION_USAGE.md](#)

Additional issues I found and fixed

1. Non-finite prediction values could still poison the optimizer

Even after the earlier non-finite candidate-parameter fix, the model prediction itself could theoretically return non-finite `z`, tyre forces, or residuals for a bad candidate. That could still make the least-squares objective become `NaN`.

I patched `evaluate_identification_prediction` so that if `solve_loads_newton`, `f_tyre`, or `f_hload` returns non-finite values during fitting, the objective returns a large finite penalty instead of `NaN`.

2. The printed "nominal parameters" could be misleading if you use `opts.model_d`, `opts.model_H`, `opts.model_q1`, or `opts.model_q2`

The earlier code always printed `veh.H`, `veh.q1`, `veh.q2`, and `veh.d`. That is fine for default simulations, but misleading if you deliberately generate clean synthetic data with a different `opts.model_d`.

I patched the identification data struct so it now records the actual simulation parameter values used by `make_model_pv`, and the output prints both:

```
Simulation parameters values: M, mt, m?, d, d0, cstea1a

MATLAB
```

This makes clean recovery tests less confusing.

Static audit checks I completed

I checked:

- the zip contents match the actual files in `/mnt/data/ident_package`;
- `my_params.m` is unchanged from your uploaded file;
- `my_model.m` only changes the intended runtime parameterization:

```
MATLAB

• all old simulation functions are still present;
• no old simulation function was removed;
• all model.f.* calls now pass the full pv;
• all solve_loads_newton calls now pass the full pv;
• the original ode15s, RK4, and ode15l branches are still present;
• the optional identification branch only runs when:
```

```
MATLAB

• the old 5-output calling style still works because the new 6th output is optional in MATLAB.
```

First tests to run

Simulation only:

```
opts = struct();
opts.run_dae = false;
opts.do_plot = true;
opts.do_identification = false;

[sim_ode15s, sim_fixed, sim_ode15l_dae, model, ctrl] = ...
```

```
MATLAB
```

Then clean identification:

```
opts = struct();
opts.run_dae = false;
opts.do_plot = true;

opts.do_identification = true;
opts.ident_source = 'ode15s';
opts.ident_fit_targets = 'Fz';
opts.ident_horizon = 3.0;
opts.ident_weight_mode = 'uniform';

[sim_ode15s, sim_fixed, sim_ode15l_dae, model, ctrl, id_result] = ...
run_oc_simple_time_sim_arbitrary('step_steer', opts);
```

Expected clean-data behavior: the free fit should be close to `[veh.H, veh.q1, veh.q2]`, and the scalar-`d` fit should be close to `veh.d`.

I still cannot claim runtime verification because I cannot run MATLAB/CasADi here. But this package has now passed the deeper static audit, and I fixed every concrete issue I found.

📄 🔍 🔗 ⬇️ 🔄 ⋮

How can I Add noise and test robustness? How can I run the code?

I'll base this on the final package options rather than guessing from memory, then give you ready-to-copy MATLAB scripts for simulation-only, clean identification, and noisy robustness tests.

已思考 25s >

Use this latest package:

[Download final rechecked package](#)

1. Put the files in your MATLAB working folder

Unzip the package, then in MATLAB:

```
MATLAB
```

Make sure these three files are in the same folder:

```
run_oc_simple_time_sim_arbitrary.m
my_model.m
my_params.m
```

```
MATLAB
```

Also make sure CasADi is on your MATLAB path, as before.

2. First run simulation only

Before testing identification, confirm the original simulation still runs:

```
clear; clc; close all;

opts = struct();
opts.run_dae = false;           % faster first test
opts.do_plot = true;
opts.do_identification = false; % important: simulation only

[sim_ode15s, sim_fixed, sim_ode15l_dae, model, ctrl] = ...
run_oc_simple_time_sim_arbitrary('step_steer', opts);

%% END
```

If this runs, then the normal simulation path is working.

3. Run clean identification, no noise

This tests whether the estimator can recover the model's own known parameters from clean self-generated data.

```
clear; clc; close all;

opts = struct();
opts.run_dae = false;
opts.do_plot = true;

opts.do_identification = true;
opts.ident_source = 'ode15s';
opts.ident_fit_targets = 'Fz';           % start with Fz only
opts.ident_horizon = 3.0;               % 3 s after steering starts
opts.ident_weight_mode = 'uniform';

[sim_ode15s, sim_fixed, sim_ode15l_dae, model, ctrl, id_result] = ...
run_oc_simple_time_sim_arbitrary('step_steer', opts);
```

After it finishes, check:

```
veh = model.veh;

fprintf('\nExpected nominal values:\n');
fprintf('veh.H = %.6f m\n', veh.H);
fprintf('veh.q1 = %.6f m\n', veh.q1);
fprintf('veh.q2 = %.6f m\n', veh.q2);
fprintf('veh.d = %.6f m\n', veh.d);

fprintf('\nIdentified free values:\n');
fprintf('H = %.6f m\n', id_result.free.H);
fprintf('q1 = %.6f m\n', id_result.free.q1);
fprintf('q2 = %.6f m\n', id_result.free.q2);

fprintf('\nIdentified scalar d:\n');
fprintf('d = %.6f m\n', id_result.scalar.d);

MATLAB
```

For clean self-generated data, you want:

```
id_result.free.H = veh.H
id_result.free.q1 = veh.q1
id_result.free.q2 = veh.q2

MATLAB
```

4. Add noise and test robustness

Fz-only noise robustness test

This is the most important first robustness test.

```
clear; clc; close all;

opts = struct();
opts.run_dae = false;
opts.do_plot = false; % faster
opts.do_identification = true;

opts.ident_source = 'ode15s';
opts.ident_fit_targets = 'Fz';
opts.ident_horizon = 3.0;
opts.ident_weight_mode = 'uniform';

% Noise settings
opts.ident_noise_enable = true;
opts.ident_noise_seed = 10;
opts.ident_noise_runs = 20; % Monte Carlo runs

opts.ident_noise_std_Fz = 50; % N, vertical load noise
opts.ident_noise_std_Fx = 0; % irrelevant for Fz-only objective
opts.ident_noise_std_Fy = 0; % irrelevant for Fz-only objective

opts.ident_plot = false;

[sin_ode15s, sin_fixed, sin_ode15l_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_arbitrary('step_steer', opts);
```

Then inspect:

```
mc = id_result.noise_monte_carlo;
veh = model.veh;

fprintf('\nNominal values:\n');
fprintf('H = %.6f m\n', veh.H);
fprintf('q1 = %.6f m\n', veh.q1);
fprintf('q2 = %.6f m\n', veh.q2);
fprintf('d = %.6f m\n', veh.d);

fprintf('\nMonte Carlo free [H,q1,q2]: mean +/- std\n');
fprintf('H = %.6f +/- %.6f m\n', mc.free_mean(1), mc.free_std(1));
fprintf('q1 = %.6f +/- %.6f m\n', mc.free_mean(2), mc.free_std(2));
fprintf('q2 = %.6f +/- %.6f m\n', mc.free_mean(3), mc.free_std(3));

fprintf('\nMonte Carlo scalar d: mean +/- std\n');
fprintf('d = %.6f +/- %.6f m\n', mc.scalar_d_mean, mc.scalar_d_std);
```

A good result is:

```
mean close to nominal value

MATLAB
```

If the standard deviation becomes large, then the manoeuvre/noise level does not identify the parameter robustly.

5. Try different noise levels

For example:

```
opts.ident_noise_std_Fz = 20; % low noise
opts.ident_noise_std_Fz = 50; % moderate noise
opts.ident_noise_std_Fz = 100; % aggressive noise

MATLAB
```

Run the same script several times and compare:

```
for i=1:10; d_end

MATLAB
```

This tells you how sensitive the fitted d , H , $q1$ and $q2$ are to vertical-load measurement noise.

6. Test Fz + Fx + Fy together

After Fz-only works, try joint tyre-force fitting:

```
clear; clc; close all;

opts = struct();
opts.run_dae = false;
opts.do_plot = false;
opts.do_identification = true;

opts.ident_source = 'ode15s';
opts.ident_fit_targets = 'FzFxFy';
opts.ident_horizon = 3.0;
opts.ident_weight_mode = 'uniform';

opts.ident_noise_enable = true;
opts.ident_noise_seed = 20;
opts.ident_noise_runs = 20;

opts.ident_noise_std_Fz = 50; % N
opts.ident_noise_std_Fx = 50; % N
opts.ident_noise_std_Fy = 50; % N

opts.ident_plot = false;

[sin_ode15s, sin_fixed, sin_ode15l_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_arbitrary('step_steer', opts);
```

Then inspect the same Monte Carlo fields:

```
id_result.noise_monte_carlo.free_mean
id_result.noise_monte_carlo.free_std
id_result.noise_monte_carlo.scalar_d_mean

MATLAB
```

My recommendation is still:

```
Fz first
FzFxFy second

MATLAB
```

7. Test immediate-vs-longer-horizon weighting

Immediate next-step emphasis:

```
opts.ident_weight_mode = 'immediate';

MATLAB
```

Full 3-second horizon:

```
opts.ident_weight_mode = 'full3sec';

MATLAB
```

Later-horizon emphasis:

```
opts.ident_weight_mode = 'later';

MATLAB
```

So a useful comparison is to run the same noisy test three times while changing only `ident_weight_mode`.

8. What to look for

After each run, check:

```
id_result.recommendation
id_result.consistency
id_result.free
id_result.scalar_d

MATLAB
```

If you get:

```
MATLAB
```

single_d_is_supported_by_this_dataset

then scalar d is acceptable for that manoeuvre.

If you get:

MATLAB

then scalar d is probably too restrictive, and you should keep separate H , q_1 , q_2 , or later fit a state-scheduled model.

Start with `step_steer`, `Fz`, `uniform`, `50 N` noise, and `20` Monte Carlo runs. That is the cleanest robustness test.

📄 🔍 🔗 ↺ ⌂ ⋮

run_oc_simple_time_sim_arbitrary(28).m
檔案

my_model(29).m
檔案

my_params(30).m
檔案

Here are my codes you provided earlier. Tell me if it can carry out the identification using the noisier data and test robustness, then try different noise level. Then test $Fz + Fx + Fy$ together. Then Test immediate-vs-longer-horizon weighting. Modify the code as you need. Then make sure I can test each option one by one. Provide me the full code then. Please also provide me the matlab commands to carry out each test one by one.

I inspected the uploaded version. It already contains the basic mechanisms for noisy targets, Monte Carlo robustness, $FzFxFy$ targets, and time weighting, but I'm going to make it easier and safer to test them one by one by adding explicit preset names, clean/noisy/sweep summary fields, and a separate MATLAB command script.

已思考 2m 38s >

Yes. Your uploaded code already had the core mechanisms for:

- noisy output identification,
- Monte Carlo robustness,
- different noise levels,
- Fz versus $Fz+Fx+Fy$,
- immediate/uniform/late horizon weighting.

through options such as `opts.ident_noise_enable`, `opts.ident_noise_runs`, `opts.ident_fit_targets`, and `opts.ident_weight_mode` in the run file `run_oc_simple_time_sim_arbitrary`. Your model is already parameterized with runtime `pv = [kappa; H; q1; q2]`, which is necessary for identification without rebuilding CasADi each time

`my_model`. Your `my_params.m` still defines the nominal values `veh.d`, `veh.H`, `veh.q1`, and `veh.q2`

`my_params`

I modified the package to make the tests easier and safer to run one by one. I added:

- clearer fit-error printing versus the actual simulation parameters,
- stored prediction SSE for the fitted predictions,
- a MATLAB script with independent sections for each test.

Download the updated package here:

Download updated identification test package

Individual files:

run_oc_simple_time_sim_arbitrary.m
my_model.m
my_params.m
run_identification_tests_one_by_one.m
README_IDENTIFICATION_TESTS.md

How to run

Unzip the package. Put these files in the same MATLAB folder:

run_oc_simple_time_sim_arbitrary.m
my_model.m
my_params.m

MATLAB

Then in MATLAB:

add_path('identified_data.m')

MATLAB

Run the script section by section using MATLAB's Run Section button.

Test order

Test 0: simulation only

This checks that the original simulator still works without identification.

MATLAB

Test 1: clean Fz -only identification

This checks whether the code can recover the model's own clean parameters:

opts.do_identification = true;
opts.ident_fit_targets = 'Fz';
opts.ident_noise_enable = false;

MATLAB

Expected result:

identified H = veh.H
identified q1 = veh.q1
identified q2 = veh.q2

MATLAB

Tests 2–4: noisy Fz robustness

These test different vertical-load noise levels:

opts.ident_noise_std_Fz = 20; % low
opts.ident_noise_std_Fz = 50; % medium

MATLAB

Each test uses:

opts.ident_noise_enable = true;
opts.ident_noise_runs = 10;

MATLAB

After your first successful run, increase:

MATLAB

or:

MATLAB

for stronger robustness statistics.

Test 5: joint $Fz + Fx + Fy$

This tests combined tyre-load and tyre-force fitting:

opts.ident_fit_targets = 'FzFxFy';

opts.ident_noise_std_Fz = 50;
opts.ident_noise_std_Fx = 30;

MATLAB

Use this only after Fz -only works.

Tests 6–8: immediate versus long-horizon weighting

Immediate emphasis:

opts.ident_weight_mode = 'immediate';

MATLAB

Uniform 3-second horizon:

MATLAB

Late-horizon emphasis:

opts.ident_weight_mode = 'late';

MATLAB

Compare the fitted values and Monte Carlo standard deviations across these three tests.

What to inspect after each test

After each section, look at:

```
id_result.free
id_result.scalar_d
id_result.error_to_simulation
id_result.consistency
id_result.recommendation

MATLAB
```

Most important robustness outputs:

```
id_result.noise_monte_carlo.free_mean
id_result.noise_monte_carlo.free_std
id_result.noise_monte_carlo.scalar_d_mean

MATLAB
```

A good noisy robustness result means:

```
free_mean close to [veh.H; veh.q1; veh.q2]
free_std small
scalar_d_mean close to veh.d

MATLAB
```

I could not run MATLAB/CasADi here, so this is still a static code update and audit. The package is set up so you can test each case independently without changing the core simulator each time.

📄 🔍 🔄 ⬇️ ⌂ ⋮ 📁 資料來源

Give me the matlab command block by block for each test.

已思考幾秒鐘 >

Below are separate MATLAB command blocks. Run them one by one. Each block is independent.

Before running, make sure MATLAB is in the folder containing:

```
run_oc_simple_time_sim_arbitrary.m
my_model.m

MATLAB
```

Test 0 — simulation only, no identification

```
clear; clc; close all;

opts = struct();

opts.run_dae = false;           % faster first test
opts.do_plot = true;
opts.do_identification = false; % simulation only

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl] = ...
    run_oc_simple_time_sim_arbitrary('step_steer', opts);
%% =====
```

Test 1 — clean Fz-only identification

```
clear; clc; close all;

opts = struct();

opts.run_dae = false;
opts.do_plot = true;

opts.do_identification = true;
opts.ident_source = 'ode15s';

opts.ident_fit_targets = 'Fz';
opts.ident_horizon = 3.0;
opts.ident_weight_mode = 'uniform';

opts.ident_noise_enable = false;
opts.ident_plot = true;

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_sim_arbitrary('step_steer', opts);

veh = model.veh;

fprintf('\nExpected nominal parameters:\n');
fprintf('veh.H = %.6f m/s', veh.H);
fprintf('veh.q1 = %.6f m/s', veh.q1);
fprintf('veh.q2 = %.6f m/s', veh.q2);
fprintf('veh.d = %.6f m/s', veh.d);

fprintf('\nIdentified free [H,q1,q2]:\n');
fprintf('H = %.6f m/s', id_result.free.H);
fprintf('q1 = %.6f m/s', id_result.free.q1);
fprintf('q2 = %.6f m/s', id_result.free.q2);

fprintf('\nIdentified scalar d:\n');
fprintf('d = %.6f m/s', id_result.scalar_d.d);
fprintf('H = %.6f m/s', id_result.scalar_d.H);

disp(id_result.consistency);
disp(id_result.recommendation);
```

Test 2 — noisy Fz robustness, low noise: 20 N

```
clear; clc; close all;

opts = struct();

opts.run_dae = false;
opts.do_plot = false;

opts.do_identification = true;
opts.ident_source = 'ode15s';

opts.ident_fit_targets = 'Fz';
opts.ident_horizon = 3.0;
opts.ident_weight_mode = 'uniform';

opts.ident_noise_enable = true;
opts.ident_noise_seed = 10;
opts.ident_noise_runs = 10;

opts.ident_noise_std_Fz = 20; % low Fz noise [N]
opts.ident_noise_std_Fx = 0;
opts.ident_noise_std_Fy = 0;

opts.ident_plot = false;

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_sim_arbitrary('step_steer', opts);

mc = id_result.noise_monte_carlo;
veh = model.veh;

fprintf('\nNominal:\n');
fprintf('H = %.6f, q1 = %.6f, q2 = %.6f, d = %.6f\n', ...
    veh.H, veh.q1, veh.q2, veh.d);

fprintf('\nMonte Carlo free [H,q1,q2], mean +/- std:\n');
fprintf('H = %.6f +/- %.6f\n', mc.free_mean(1), mc.free_std(1));
fprintf('q1 = %.6f +/- %.6f\n', mc.free_mean(2), mc.free_std(2));
fprintf('q2 = %.6f +/- %.6f\n', mc.free_mean(3), mc.free_std(3));

fprintf('\nMonte Carlo scalar d, mean +/- std:\n');
fprintf('d = %.6f +/- %.6f\n', mc.scalar_d_mean, mc.scalar_d_std);
```

Test 3 — noisy Fz robustness, medium noise: 50 N

```
clear; clc; close all;

opts = struct();

opts.run_dae = false;
opts.do_plot = false;

opts.do_identification = true;
opts.ident_source = 'ode15s';

opts.ident_fit_targets = 'Fz';
opts.ident_horizon = 3.0;
opts.ident_weight_mode = 'uniform';

opts.ident_noise_enable = true;
opts.ident_noise_seed = 20;
opts.ident_noise_runs = 10;

opts.ident_noise_std_Fz = 50; % medium Fz noise [N]
opts.ident_noise_std_Fx = 0;
opts.ident_noise_std_Fy = 0;

opts.ident_plot = false;

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_sim_arbitrary('step_steer', opts);

mc = id_result.noise_monte_carlo;
veh = model.veh;

fprintf('\nNominal:\n');
fprintf('H = %.6f, q1 = %.6f, q2 = %.6f, d = %.6f\n', ...
    veh.H, veh.q1, veh.q2, veh.d);

fprintf('\nMonte Carlo free [H,q1,q2], mean +/- std:\n');
fprintf('H = %.6f +/- %.6f\n', mc.free_mean(1), mc.free_std(1));
fprintf('q1 = %.6f +/- %.6f\n', mc.free_mean(2), mc.free_std(2));
fprintf('q2 = %.6f +/- %.6f\n', mc.free_mean(3), mc.free_std(3));

fprintf('\nMonte Carlo scalar d, mean +/- std:\n');
fprintf('d = %.6f +/- %.6f\n', mc.scalar_d_mean, mc.scalar_d_std);
```

Test 4 — noisy Fz robustness, high noise: 100 N


```
opts.idnt_plot = false;

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_sim_arbitrary('step_steer', opts);

mc = id_result.noise_monte_carlo;

fprintf('\nlte weighting result:\n');
fprintf('free mean [H q1 q2] = [%.6f %.6f %.6f]\n', ...
    mc.free_mean(1), mc.free_mean(2), mc.free_mean(3));
fprintf('free std [H q1 q2] = [%.6f %.6f %.6f]\n', ...
    mc.free_std(1), mc.free_std(2), mc.free_std(3));
fprintf('scalar d mean/std = %.6f / %.6f\n', ...
    mc.scalar_d_mean, mc.scalar_d_std);
```

Test 9 — compare different steering manoeuvres

After `step_steer` works, repeat with `sine_steer` :

```
clear; clc; close all;

opts = struct();

opts.run_dae = false;
opts.do_plot = false;

opts.sine_mag = deg2rad(8.0);
opts.sine_freq = 0.5;
opts.sine_tau = 0.4;

opts.do_identification = true;
opts.idnt_source = 'ode15s';

opts.idnt_fit_targets = 'Fz';
opts.idnt_horizon = 3.0;
opts.idnt_weight_mode = 'uniform';

opts.idnt_noise_enable = true;
opts.idnt_noise_seed = 80;
opts.idnt_noise_runs = 10;

opts.idnt_noise_std_Fz = 50;
opts.idnt_noise_std_Fx = 0;
opts.idnt_noise_std_Fy = 0;

opts.idnt_plot = false;

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_sim_arbitrary('sine_steer', opts);

mc = id_result.noise_monte_carlo;

fprintf('\nsine steer Fz robustness result:\n');
fprintf('free mean [H q1 q2] = [%.6f %.6f %.6f]\n', ...
    mc.free_mean(1), mc.free_mean(2), mc.free_mean(3));
fprintf('free std [H q1 q2] = [%.6f %.6f %.6f]\n', ...
    mc.free_std(1), mc.free_std(2), mc.free_std(3));
fprintf('scalar d mean/std = %.6f / %.6f\n', ...
    mc.scalar_d_mean, mc.scalar_d_std);
```

Test 10 — stronger Monte Carlo after everything works

Only do this after the previous tests work, because this is slower.

```
clear; clc; close all;

opts = struct();

opts.run_dae = false;
opts.do_plot = false;

opts.do_identification = true;
opts.idnt_source = 'ode15s';

opts.idnt_fit_targets = 'Fz';
opts.idnt_horizon = 3.0;
opts.idnt_weight_mode = 'uniform';

opts.idnt_noise_enable = true;
opts.idnt_noise_seed = 100;
opts.idnt_noise_runs = 50; % stronger Monte Carlo, slower

opts.idnt_noise_std_Fz = 50;
opts.idnt_noise_std_Fx = 0;
opts.idnt_noise_std_Fy = 0;

opts.idnt_plot = false;

[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl, id_result] = ...
    run_oc_simple_time_sim_arbitrary('step_steer', opts);

mc = id_result.noise_monte_carlo;
veh = model.veh;

fprintf('\nfinal stronger robustness test:\n');
fprintf('Nominal H/q1/q2/d = %.6f / %.6f / %.6f / %.6f\n', ...
    veh.H, veh.q1, veh.q2, veh.d);

fprintf('free mean [H q1 q2] = [%.6f %.6f %.6f]\n', ...
    mc.free_mean(1), mc.free_mean(2), mc.free_mean(3));
fprintf('free std [H q1 q2] = [%.6f %.6f %.6f]\n', ...
    mc.free_std(1), mc.free_std(2), mc.free_std(3));
fprintf('scalar d mean/std = %.6f / %.6f\n', ...
    mc.scalar_d_mean, mc.scalar_d_std);
```

Start with Test 0, then Test 1, then Test 3. If those work, continue with the rest.

📄 🔍 🔄 ⬇️ 🔄 ⋮

Summarize the commands block by block and send them to me in a .txt file

已思考幾秒鐘 >

I created the .txt file with the MATLAB commands block by block:

Download the MATLAB command file ↗

📄 🔍 🔄 ⬇️ 🔄 ⋮

- matlab_identification_commands_bloc...
文件
- my_model(31).m
檔案
- my_params(40).m
檔案
- README_IDENTIFICATION_TESTS(1).md
檔案
- run_identification_tests_one_by_one(1)...
檔案
- run_oc_simple_time_sim_arbitrary(30).m
檔案

Check these codes and commands to make sure the codes are implemented correctly as intended and the commands will work correctly too.

I'll check the actual uploaded files, not just the snippets: the run file, model file, params file, command text, and the one-by-one script. I'll look for MATLAB syntax issues, undefined fields/functions, option mismatches, and whether the commands match the implemented outputs.

The main simulator/model implementation is mostly consistent: `my_model.m` uses `pv=[kappa;H;q1;q2]` , and the run file passes that full vector through the simulation, post-processing, Newton load solve, DAE residual, and identification evaluations. I found one practical problem in the command text: it is fine as a human-readable .txt , but not valid if you paste/run the whole file as a MATLAB script because the headings are not comments.

已思考 3m 16s >

I checked the uploaded files and commands. The core implementation is consistent with the intended identification workflow, but I found one practical command-file issue and fixed it in a checked package.

Result

The simulator/model side is implemented as intended:

- `my_model.m` correctly uses runtime identification parameters:

MATLAB
so identification can change `H` , `q1` , and `q2` without rebuilding the CasADi graph. `my_model`

- `run_oc_simple_time_sim_arbitrary.m` keeps simulation-only behavior when:

MATLAB
and exposes the intended options for `Fz` , `FxFy` , `FzFxFy` , noise, Monte Carlo runs, and time weighting.
`run_oc_simple_time_sim_arbitrary`

- `my_params.m` still defines the nominal physical values:

MATLAB
so the clean self-generated identification test has a known reference. `my_params`

Issue I found

The .txt command file is fine for copying one code block at a time, but it is not valid as a whole MATLAB script because headings like:

```
=====
TEST 0 - Simulation only, no identification
=====
```

are not MATLAB comments. `matlab_identification_commands_...`

So I created a checked version where headings are converted to MATLAB comments / `%` sections.

Updated checked files

[Download checked package](#)

Individual files:

[run_oc_simple_time_sim_arbitrary.m](#)
[my_model.m](#)
[my_params.m](#)
[run_identification_tests_one_by_one_checked.m](#)
[matlab_identification_commands_block_by_block_checked.m](#)
[matlab_identification_commands_block_by_block_checked.txt](#)
[STATIC_CHECK_REPORT.txt](#)

What to use

Use this file for MATLAB section-by-section testing:

```
MATLAB
```

or:

```
MATLAB
```

Run sections in this order:

```
Test 0 -> Test 1 -> Test 3 -> Tests 2/4/5 -> Tests 6/7/8 -> Tests 9/10
```

Final caveat

I still cannot execute MATLAB/CasADi here, so this is a static code and command audit, not a runtime test. But I checked the actual uploaded files directly, verified the option names and output fields used by the commands, and fixed the command-file/script usability issue.

[📄](#) [🔗](#) [🔍](#) [🔒](#) [🔄](#) [⋮](#) [📄](#) [資料来源](#)

Do you think it is logic and making sense of not if I plan to use Carsim data for such identification? Moreover, to implement the vehicle model in NMPC in order for the model to predict tyre load or even forces as best as possible (for example, from Carsim co-simulation), what should we do about the the current identification?

This is an important modelling decision, not just a coding question. I'll treat the current identification code as a clean-data prototype and explain how it should be upgraded before trusting CarSim-calibrated parameters inside NMPC.

已思考 15s >

Yes, it is logical and worth doing to use CarSim data for this identification, but only with the right interpretation.

The key point is:

With CarSim data, H , q_1 , q_2 , d should be treated as effective prediction parameters, not pure physical geometry.

Your current model has already been prepared in the right direction because `my_model.m` uses runtime parameters

```
MATLAB
```

so the same CasADi model can evaluate different H , q_1 , and q_2 values without rebuilding the model graph. `my_model` Your nominal parameters still come from

```
veh.d = veh.h * 0.7;  
veh.H = veh.h - veh.d;  
veh.q1 = veh.d;  
MATLAB
```

so the clean simulation test has a known reference. `my_params`

1. Does using CarSim data make sense?

Yes, but the goal should not be "find the true physical d ."

The goal should be:

find the H , q_1 , q_2 or d that make your NMPC model predict CarSim tyre loads/forces best.

CarSim has much richer effects than your current reduced model, for example:

- nonlinear suspension kinematics,
- roll-center migration,
- pitch-center effects,
- anti-roll-bar distribution,
- compliance,
- camber effects,
- tyre relaxation,
- load-sensitive tyre behaviour,
- combined-slip behaviour,
- possible aero/downforce distribution,
- actuator filtering,
- possibly different definitions of wheel force frames.

Your current model cannot reproduce all of these exactly with one scalar d . Therefore, if you fit d to CarSim, the identified d is really an effective equivalent load-transfer parameter.

That is still useful for NMPC because NMPC does not need every parameter to be physically pure. It needs the prediction model to be accurate enough over the control horizon.

2. What should the current identification be used for?

Your current identification should be used in three stages.

Stage A — Clean self-generated data

This is what the current code is mainly designed for.

Purpose:

Check whether the identification algorithm can recover known parameters from its own model.

Expected result:

$$\begin{aligned}\hat{H} &\approx veh.H \\ \hat{q}_1 &\approx veh.q1 \\ \hat{q}_2 &\approx veh.q2 \\ \hat{d} &\approx veh.d\end{aligned}$$

If this fails, then the identification implementation or test excitation is wrong.

Stage B — Noisy self-generated data

This tests robustness.

Purpose:

Check whether the manoeuvre contains enough information to identify H , q_1 , q_2 , d .

This is exactly what your noise and Monte Carlo options are for:

```
opts.ident_noise_enable = true;  
nmqc.ident_noise_runc = 10;  
MATLAB
```

The current run file already has options for target type, horizon weighting, noise level, and Monte Carlo robustness. `run_oc_simple_time_sim_arbitrary`

If the identified values become unstable with moderate noise, then the problem is not yet ready for CarSim data.

Stage C — CarSim data

Only after Stage A and B work should you use CarSim.

For CarSim, you should not immediately fit scalar d only. Fit:

$$H, q_1, q_2$$

first. Then test whether the scalar relation

$$q_1 \approx q_2$$

and

$$H \approx h - q_1$$

holds.

If it holds, then scalar d is acceptable.

If it does not hold, then scalar d is too restrictive, and you should keep H , q_1 , and q_2 separate for NMPC.

3. What should we identify from CarSim first?

The best order is:

F_z first, then $F_x + F_y + F_{y^*}$, then trajectory prediction.

Step 1 — Fit tyre vertical loads only

Use CarSim wheel loads:

$$F_{z,fl}^{CS}, F_{z,fr}^{CS}, F_{z,rl}^{CS}, F_{z,rr}^{CS}$$

and minimize:

$$J_z = \sum_i \frac{\hat{F}_{z,i}(H, q_1, q_2) - F_{z,i}^{CS}}{F_{z,scale}}^2$$

This is the cleanest first target because q_1 and q_2 directly control lateral load transfer in your load equations.

Step 2 — Validate with tyre forces

After fitting F_z , check whether the resulting model predicts:

$$F_x, F_y$$

reasonably well.

But do not immediately force d to compensate for bad F_x/F_y , because force mismatch may come from the tyre model, not load transfer.

Step 3 — Jointly fit $F_z + F_x + F_y$

Only after F_z -only fitting works, use:

MATLAB

This is useful for NMPC because NMPC cares about both tyre loads and tyre forces.

But the weighting must be normalized. Otherwise the biggest-magnitude channel can dominate. Your options already include scaling factors such as:

```
opts.ident_Fz_scale = veh.m*veh.g/lz;  
opts.ident_Fx_scale = veh.m*veh.v/dx;
```

MATLAB

That is the right idea.

4. What should change before using real CarSim data?

Your current code identifies from its own simulated `sim_ode15s`, `sim_fixed`, or `sim_ode15i_dae` result. For CarSim, you need a new data-import layer.

You need a structure like:

```
data_cs.t  
data_cs.x_scaled  
data_cs.u_scaled  
data_cs.Fz  
data_cs.Fx  
data_cs.Fy
```

MATLAB

or in physical form:

```
data_cs.t  
data_cs.x_phys  
data_cs.u_phys  
data_cs.Fz_phys  
data_cs.Fx_phys  
data_cs.Fy_phys
```

MATLAB

Then the identification function should run exactly the same objective, but with CarSim target data instead of self-generated `sim_ode15s`.

The important preprocessing checks are:

Item	Must match
Time	CarSim and MATLAB data must be synchronized
Units	N, Nm, rad, m/s, rad/s
Steering	road-wheel angle versus steering-wheel angle
Torque	total axle torque versus per-wheel torque
Brake sign	your model uses braking torque as non-positive
Force frame	wheel frame versus vehicle/body frame
Vertical load sign	positive downward/upward convention must match
State definition	your $u_x, u_y, r, \phi, \dot{\phi}$, ϕ_{hidot} , θ , $\dot{\theta}$ must correspond to CarSim channels
Position point	your model uses VIP / vehicle invariant point, not necessarily CarSim CG output
Road	flat road or same curvature κ input

If these are not aligned, the optimizer will identify a fake d to hide frame/sign/unit errors.

5. For NMPC, should we use scalar d , free $H/q1/q2$, or scheduled parameters?

For NMPC, I would not jump directly to arbitrary time-varying $d(t)$.

Bad NMPC form

$$d = d(t)$$

This is bad because NMPC does not know the future time-varying $d(t)$ unless you prescribe it.

Better NMPC form

Use constant calibrated parameters first:

$$H = H_0, \quad q_1 = q_{10}, \quad q_2 = q_{20}$$

This is the simplest NMPC-ready model.

Even better form

Use state-scheduled parameters:

$$\begin{aligned} q_1 &= q_1(a_y, \phi, \dot{\phi}, u_x, \delta) \\ q_2 &= q_2(a_y, \phi, \dot{\phi}, u_x, \delta) \\ H &= H(a_x, a_y, \phi, \theta) \end{aligned}$$

For example:

$$\begin{aligned} q_1 &= q_{10} + q_{1a}|a_y| + q_{1\phi}\phi + q_{1\dot{\phi}}\dot{\phi} \\ q_2 &= q_{20} + q_{2a}|a_y| + q_{2\phi}\phi + q_{2\dot{\phi}}\dot{\phi} \end{aligned}$$

This is much more useful than free $d(t)$ because NMPC can evaluate it over the future prediction horizon.

6. What should we do with the current identification?

My recommendation is to use the current identification in this exact sequence:

Phase 1 — Validate code

Run:

```
opts.ident_fit_targets = 'Fz';  
opts.ident_noise_enable = false;
```

MATLAB

Expected:

$$\hat{H}, \hat{q}_1, \hat{q}_2, \hat{d}$$

recover the nominal simulation values.

Phase 2 — Robustness with synthetic noise

Run 20 N, 50 N, 100 N noise.

If the estimated standard deviation is small, the manoeuvre is informative.

If not, improve the manoeuvre before using CarSim.

Good manoeuvres are:

- step steer,
- sine steer,
- ramp steer,
- increasing steering amplitude,
- different speeds,
- left and right steering,
- combined drive/brake plus steer later.

Phase 3 — CarSim Fz-only identification

Import CarSim:

$$x^{CS}, u^{CS}, F_z^{CS}$$

Fit:

$$H, q_1, q_2$$

Do not fit scalar d first.

Phase 4 — CarSim validation with Fx/Fy

With fitted H, q_1, q_2 , evaluate:

$$\hat{F}_x, \hat{F}_y$$

against CarSim.

If F_z is good but F_x/F_y is bad, then the problem is likely the tyre model, not d .

Phase 5 — Joint CarSim Fz+Fx+Fy fitting

Only then fit:

MATLAB

But keep an eye on whether H, q_1, q_2 move far away from the F_z -only values. If they do, it means $H/q_1/q_2$ are being used to compensate tyre-model errors.

Phase 6 — NMPC prediction validation

Finally, check short rollout prediction:

$$x_0^{CS}, u_{0:N}^{CS} \rightarrow \hat{x}_{1:N}, \hat{F}_{x,1:N}, \hat{F}_{x,1:N}, \hat{F}_{y,1:N}$$

